hCaptcha (Intuition Machines) — client-side assessment (2026-10-31)

Subject: hCaptcha's web widget + anti-bot stack
Method: public-asset analysis + runtime observation of the public demo widget. All hCaptcha API calls were **read-only observations or fulfilled locally** — we did **not** solve a challenge, forge a token, or touch any customer/prod traffic.
Result: hCaptcha is **architecturally different from every device-intel SDK in this project** — its client is a **server-orchestrated, JWT-ticketed, hash-pinned, WASM-backed module that refuses to run under automation at all**.

> **TL;DR — the client is not the boundary, and it's not even the attack surface.** hCaptcha's server issues signed JWT tickets (`hsw` / `hsj`) that point to **hash-pinned** module URLs; the anti-bot logic lives in a **context-gated WASM module** (`hsw.js`) whose strings don't decode and whose WASM never instantiates without a valid server-issued ticket + human-like session. In our automated runs the widget rendered but the first server contact returned **"Please try again"** — the gate fired *before any challenge was served*. Unlike Stytch/Oscilar/Verisoul/Castle/TikTok, we could not read the payload, could not mint a token, and could not tamper a session.

---

1. Deployment / asset surface

Asset	Size	Role
`js.hcaptcha.com/1/api.js`	415 KB	loader — exposes `window.hcaptcha` (`render/execute/reset/getResponse/…`)
`…/static/hcaptcha.html`	678 KB	challenge shell (inline JS: orchestration, canvas rendering)
`…/static/hsw.js`	1.5 MB	the **secure module** (anti-bot/WASM wrapper)
`…/static/challenge.js`	45 KB	image-label (`image_label_area_select`) challenge UI
`…/static/i18n/*`	—	locale tables
`imgs3.hcaptcha.com`	—	challenge images
`*.w.hcaptcha.com`	—	per-widget subdomain (logo/beacon)
`api.hcaptcha.com`	—	`checksiteconfig` , `getcaptcha` , `checkcaptcha`
`accounts.hcaptcha.com/demo`	—	public demo widget (sitekey `a5f74b19-…`)

2. The orchestration (observed)

```
page → api.js → window.hcaptcha.render(sitekey)
  → POST api.hcaptcha.com/checksiteconfig?v=<hash>&host=…&sitekey=…
       ← { features:{enc_get_req:true,andint:true},
           c:{ type:"hsw", req:<JWT> }, pass:true }
  → load <JWT.l>  (hash-pinned hsw.js)   [JWT.i = sha256 of the bundle]
  → POST api.hcaptcha.com/getcaptcha/<sitekey>
       ← { c:{ type:"hsj", req:<JWT> } }  (challenge ticket; body encrypted)
  → render challenge (challenge.js) → user solves → submit answer
  → POST /checkcaptcha → server verifies → issue token
```

The JWT tickets (HS256, decoded):

```
checksiteconfig → c.req (type hsw):
  header: {typ:"JWT", alg:"HS256"}
  payload: { f:0, s:4, t:"w",
             d:"<296-char encrypted blob>",
             l:"/c/fcf9b503…/hsw.js",          ← module path
             i:"sha256-ENe4g+VA5x5yz50Z…=",     ← module integrity (sha256)
             e:1793451173,                      ← expiry
             n:"hsw", c:1000 }

getcaptcha → c.req (type hsj):  same shape, n:"hsj", t:"j", d encrypted.
```

- **Every module load is bound to a server-issued, expiring, signed ticket.**
- The ticket pins the **exact bundle hash** (`i`) — the client is told *which* hsw.js to run and its expected SHA-256, so serving a modified module is detectable server-side.
- The `d` payload (config/session binding) is **encrypted**; the getcaptcha body is **encrypted** (`enc_get_req: true`). Even the challenge data is opaque on the wire.

3. The secure module (hsw.js) — context-gated WASM

`hsw.js` (1.5 MB) is a JS wrapper around an **embedded WASM binary**:

- `anF` = WASM import table, `aep()` = WASM instance, `aIW(n)` = string decoder, `_K(...)` = native-call bridge.
- Multi-layer string protection: custom-alphabet base64 (`Li`) → string table (`$h`) → **XOR decode** (`aIW`, const `1112731091` + murmur-mix) → UTF-8.
- An **~80 KB encrypted blob** (`0x1a1b1c1d…` header) is fed through `_A(anr, …)` / `_K(…)` string decodes — the WASM is assembled from encrypted table entries, not shipped as a plain `.wasm`.
- **Context gating:** with the real JWT tickets and the real widget, the strings still never decode and `WebAssembly.instantiate` never fires in our automated runs. The module is designed so its internals are only released inside a session the server has already vouched for.

4. The gate is server-side, and it fires early

In our runs (headless and headed Playwright), the demo widget rendered — the "I am human" checkbox (`#checkbox`, 30×30) was present and interactable — but

the **challenge iframe already showed "Please try again ⚠️ / Verify"** on first contact. Consequences, observed:

- `getcaptcha` never produced a solvable challenge for the checkbox flow.
- `hsw.js` loaded (per ticket) but its WASM never initialized, strings never decoded.
- `window.hcaptcha.execute()` fired but returned no token.

Interpretation (honest): the assessment that "this is an automated browser" happens **server-side on the first interaction** (via request fingerprinting, TLS/JA3, behavior, and whatever the initial page sends) — *before* the client-side anti-bot module is even allowed to run. The client WASM is a *second* layer that would run in a trusted session; it is not the first gate and not the boundary.

5. Honest boundaries

- We did **not** read the hsw WASM internals (string table / bytecode) — the module is context-gated and never initialized for us. Reversing it would require a real, trusted, human-like session (or breaking the JWT/encryption layers first), which is out of scope and out of reach from automation.
- We did **not** solve any challenge, mint any token, or verify against `siteverify`. The getcaptcha/checkcaptcha calls were read-only or locally fulfilled.
- We did **not** contact any customer/prod hCaptcha deployment — only the public demo widget on `accounts.hcaptcha.com`.

6. How it compares (cross-vendor)

	Stytch/Oscilar/Verisoul/Castle/TikTok	hCaptcha
Payload readable client-side?	**Yes** (we read all of them)	**No** — encrypted on wire, WASM context-gated
Token mintable client-side?	**Yes** (all minted/tampered)	**No** — requires server-issued tickets + challenge solve
Client anti-bot runs under automation?	Yes (that's why we could RE it)	**No — refuses to initialize**
Server-issued per-request state?	Partial (tid/rids, nonces)	**Full — JWT tickets + hash-pinned bundles + encrypted channel**
Real boundary	server-side scoring	server-side assessment + human challenge

Placement: on "resistance to a determined RE with runtime control," hCaptcha is the **strongest client of the batch — ~85** — but for a different reason than the device-intel vendors: not because its client is obfuscated harder (it is), but because **the client is not where the decision happens**. The proof of humanity is a server-verified challenge; the client module is a ticketed, hash-pinned, context-gated second layer. This is closer to Fingerprint Pro's "server-computed identity" architecture than to the client-mints-everything SDKs.

7. What this means for defenders/attackers

- **For defenders:** this is the right shape. Per-request JWT tickets + hash-pinned bundles + encrypted challenge data + server-side session assessment = the client can't be repurposed the way a device-intel SDK can. The residual risk is the challenge itself (solver farms / ML solvers), which is a **computer-vision + behavioral problem, not an RE problem**.
- **For attackers:** the cost moved from "read/tamper the client" to "solve the challenge + look human enough server-side" — a fundamentally harder, higher-cost problem that no client-side red team in this project faced.

8. Artifacts

File	What
`assets/hsw.js`	the 1.5 MB secure module (from CDN)
`assets/hcaptcha.html`	678 KB challenge shell
`assets/challenge.js`	45 KB image-label challenge UI
`scripts/hc-capture*.mjs`, `hc-live.mjs`	widget load + asset capture
`scripts/hc-flow.mjs`, `hc-getcaptcha*.mjs`	read-only API observation
`scripts/hc-hsw*.mjs`, `hc-widget-trace.mjs`	hsw aIW patch attempts (context-gated)
`scripts/hc-execute.mjs`, `hc-gate.mjs`	execute() + gate observation
`artifacts/getcaptcha-full.json`	checksiteconfig/getcaptcha tickets (decoded)
`artifacts/widget-shot.png`	screenshot of the gated widget